

Senior+ Full Stack Technical Test

Customer Management Application

The Brief

Build a simple full-stack application to manage customer records.

- Create a Spring Boot application
- Expose an HTTP endpoint to post customer details (First Name, Last Name, Date of Birth)
- Use an in-memory database to persist customer details
- Create an HTTP endpoint to retrieve the customer details
- Build a simple UI to create and display customers

Time Constraint

3 – 4 hours (typical submissions take 3 – 5 hours).

This is a guideline, not a hard limit. We value quality over speed — spend the time you need to produce work you're proud of, but be mindful of scope.

Technology Preferences

You are free to choose from the following. You will **not** be penalised for choosing an acceptable alternative.

Backend Language: Kotlin (preferred), Java | *Acceptable: Python*

Backend Framework: Spring Boot 3.x | *Acceptable: FastAPI*

Frontend: React 19 + TypeScript | *Acceptable: Other modern frameworks with justification*

Database: H2 (in-memory) | *Acceptable: Any embedded / in-memory database*

AI Usage Documentation

You **must** include an `AI_USAGE.md` file documenting:

- Which AI tools you used (GitHub Copilot, ChatGPT, Claude, etc.)
- What tasks you delegated to AI vs wrote yourself
- How you validated AI-generated code

- Examples where you corrected AI mistakes
- Approximate time breakdown with / without AI assistance
- How AI impacted your development process

Submission

Please submit your solution as a **git bundle** file. This preserves your full commit history, which we review as part of the evaluation.

To create a git bundle from your repository:

```
git bundle create your-name-tech-test.bundle --all
```

See <https://git-scm.com/docs/git-bundle> for details.

What We're Looking For

Your submission should be as close to production quality as possible. Treat this as code you would be comfortable shipping to production and maintaining long-term. We are evaluating not just whether it works, but whether it demonstrates the engineering standards expected at your level.

- A working end-to-end solution (backend + frontend + persistence)
- Clean, well-structured code that's easy to follow
- Production-quality error handling, validation, and edge case coverage
- Sensible assumptions where the brief is deliberately vague
- Appropriate test coverage (unit, integration, and/or E2E as you see fit)
- Clear documentation (README with setup instructions and design decisions)
- Thoughtful use of AI tools, with evidence of critical review

Good luck! If you have any questions about the brief, don't hesitate to ask.